omeru.

INTERNAL — CONFIDENTIAL

Data Strategy & Analytics Recommendation

A full audit of OmeruHQ's current data model, what's missing for analytical readiness, the KPIs that will matter as the platform scales, and a prioritised implementation roadmap for turning raw Postgres tables into a data scientist's operating environment.

DOCUMENT	DATE	SCOPE
DS-001 · Data Strategy	June 2026	hq.omeru.io + Omeru backend (Supabase / Prisma)

01

Executive Summary

Where we are, what the data reveals, and what to do first.

OmeruHQ has a **well-structured transactional database** — orders, bookings, merchants, customers, products, and audit logs are all modelled cleanly in Prisma/PostgreSQL. However, the schema is optimised for *running the business*, not yet for *understanding it*. A data scientist dropped into the current database today would be able to answer "how much did merchant X earn last month?" but could not answer "why are customers dropping off?" or "which category acquires the stickiest customers?"

This document identifies the gaps, proposes specific new tables and columns, defines the KPI framework, and provides SQL views that make the existing data immediately more useful.

TABLES ANALYSED	ANALYTICS GAPS FOUND	KPIS DEFINED	SQL VIEWS PROPOSED
13	**11**	**24**	**5**
Prisma models + invite_applications	Missing events, denorm issues, no funnel	Across merchant, customer, platform layers	Ready to run against existing schema

> **Highest-value action:** Add a `platform_events` table and begin logging product views, search queries, and cart adds. Without this, the conversion funnel is completely invisible — we can see orders but not what happened before them.

What we can already measure (no changes needed)

✓ CURRENTLY TRACKABLE

- GMV per merchant per period
- Average order value (AOV) per merchant
- Order completion rate (COMPLETED / total)
- Top products by revenue
- Unique customers per merchant
- Booking completion & rejection rates
- Wishlist saves per product
- Merchant KYC completion funnel
- Invite application volume by province & category
- Repeat customer rate (customers with >1 order)
- Platform fee revenue (via PlatformBranding.platform_fee × GMV)
- Merchant opt-in to service features

✗ CURRENTLY INVISIBLE

- Product view → cart → order funnel
- Cart abandonment rate
- Search queries & zero-result searches
- Time-on-bot / session depth
- Customer geographic distribution
- Order status transition durations
- Refund / cancellation reasons
- Cross-merchant shopping behaviour
- Merchant catalogue health (stale products)
- WhatsApp message volume per session
- UTM / referral attribution for merchants

02 Current Schema Audit

Table-by-table assessment of analytical readiness.

TABLE	ANALYTICAL VALUE	STATUS	KEY GAP
Order	GMV, AOV, conversion, cohorts	PARTIAL	No status transition timestamps — can't measure fulfilment speed
OrderItem	Product revenue attribution, basket analysis	GOOD	No promotion/discount field
Merchant	Cohort analysis, category performance, retention	PARTIAL	province is missing as a field; address is a free-text string
MerchantCustomer	Retention, repeat rate, churn	PARTIAL	No province, no order_count denorm, last_interaction_at is the only behavioural signal
UserSession	Session flow, cart abandonment, state machine	WEAK	cart_json is a raw string — not queryable. No session timestamps or step counts
Product	Catalogue health, out-of-stock patterns	PARTIAL	No view count, no search impression count
Wishlist	Demand signals, purchase intent	GOOD	No conversion tracked (wishlist → order)
Booking	Service demand, merchant capacity	GOOD	No no-show tracking, no rebooking rate
AuditLog	Admin actions, compliance, debugging	GOOD	metadata_json not indexed — slow for aggregate queries
invite_applications	Pipeline, acquisition attribution, market research	PARTIAL	No link back to Merchant record after activation — pipeline not closed-loop
ShortLink	Attribution, campaign tracking	WEAK	No click count, no referrer, no UTM — links are untracked
MerchantInvite	Team growth, multi-owner patterns	GOOD	Minor: no expiry on pending invites
ServiceApplication	Service expansion pipeline	GOOD	No time-to-decision tracking

Critical: UserSession.cart_json is a black box

The most valuable pre-conversion signal — what was in the cart before a customer abandoned — is stored as an unstructured JSON string. It cannot be aggregated, joined, or used for cohort analysis without parsing in application code. This needs to be normalised into a `CartItem` table or the field changed to `jsonb` with a GIN index.

> **Immediate action required:** Alter `UserSession.cart_json` column type from `text` to `jsonb` in Postgres. This enables `cart_json ->> 'product_id'` extraction, indexing, and aggregate queries without any application code change.

ShortLink has zero attribution data

Every `ShortLink` has a `code` and a `url` — nothing else. There is no click counter, no referrer header capture, no UTM passthrough. These links are being shared (presumably in WhatsApp, social, or by merchants) but we have no idea who is clicking them or which campaigns drive traffic. Adding just two columns — `click_count int` and `last_clicked_at timestamptz` — plus a server-side increment would unlock link-level attribution.

03 KPI Framework

24 metrics across three layers: platform, merchant, and customer.

Platform-Level KPIs

These are the top-of-house numbers — what the business reports and what determines platform health.

KPI	FORMULA	SOURCE TABLES	CADENCE
Gross Merchandise Value (GMV)	SUM(order.total) WHERE status = COMPLETED	Order	Daily / Weekly / Monthly
Platform Revenue	GMV × platform_fee %	Order, PlatformBranding	Monthly
Active Merchants (MAM)	COUNT(DISTINCT merchant_id) with ≥1 COMPLETED order in 30d	Order	Weekly
Active Customers (MAC)	COUNT(DISTINCT customer_id) with ≥1 COMPLETED order in 30d	Order	Weekly
Order Completion Rate	COMPLETED orders / total orders	Order	Weekly
Merchant Pipeline Conversion	Activated merchants / invite_applications received	invite_applications, Merchant	Monthly
Merchant Retention Rate	% merchants active in month N who are also active in month N+1	Order, Merchant	Monthly
Avg Time Invite → First Sale	AVG(first order.createdAt - merchant.createdAt)	Order, Merchant	Monthly

Merchant-Level KPIs

Per-merchant health scores, used for merchant success outreach and product decisions.

KPI	FORMULA	SOURCE TABLES
Average Order Value (AOV)	GMV / completed order count	Order
Customer Repeat Rate	Customers with >1 order / total unique customers	Order, MerchantCustomer
Catalogue Efficiency	GMV / active product count	Order, Product
Out-of-Stock Rate	Products where is_in_stock = false / total products	Product

KPI	FORMULA	SOURCE TABLES
Booking Rejection Rate	REJECTED bookings / total bookings	Booking
Wishlist Demand Score	COUNT(wishlist) per product — proxy for unmet demand	Wishlist, Product
Category Mix	Revenue breakdown by store_category	Order, Merchant
KYC Completion Rate	Merchants with kyc_online_completed = true / total merchants	Merchant

Customer-Level KPIs

Customer health and value signals, used for retention, referral, and experience improvement.

KPI	FORMULA	SOURCE TABLES
Customer Lifetime Value (CLV)	SUM(order.total) per customer_wa_id, COMPLETED only	Order
Time to Second Order	AVG(2nd order.createdAt - 1st order.createdAt) per customer	Order
Cross-Merchant Shopping Rate	Customers who ordered from ≥2 distinct merchants	Order
Opt-out Rate	MerchantCustomers with opt_out = true / total	MerchantCustomer
Cart Abandonment Rate	Sessions with cart_json ≠ null AND no resulting Order	UserSession, Order NEEDS JSONB FIX
Wishlist-to-Purchase Rate	Products bought that were previously wishlisted by same wa_id	Wishlist, OrderItem, Order
Session Depth	Number of state transitions per session	AuditLog, UserSession NEEDS LOGGING
Province Distribution	Customer count by province	MerchantCustomer COLUMN MISSING

04 What to Add

New tables, columns, and indexes that unlock the invisible funnel.

P1 — CRITICAL platform_events — Behavioural event log

Right now the most important question — *"how many people viewed this product but didn't buy it?"* — is completely unanswerable. A single lightweight events table changes this. Log everything the bot handles: product views, search queries, menu navigations, cart adds, cart removes. Keep it append-only; never update rows.

```
CREATE TABLE platform_events (
  id           uuid         DEFAULT gen_random_uuid() PRIMARY KEY,
  created_at   timestamptz  DEFAULT now(),
  event_type   text         NOT NULL,   -- 'product_view' | 'search' | 'cart_add'
                                        --  'cart_remove' | 'menu_browse' | 'wa_share'
  session_wa_id text,                   -- anonymised if needed
  merchant_handle text,
  product_id   text,
  search_query text,
  metadata     jsonb                     -- catch-all for future fields
);

-- Indexes for common analytical query patterns
CREATE INDEX idx_events_type_date    ON platform_events (event_type, created_at);
CREATE INDEX idx_events_merchant     ON platform_events (merchant_handle, created_at);
CREATE INDEX idx_events_product      ON platform_events (product_id) WHERE product_id IS NOT NUI
CREATE INDEX idx_events_session      ON platform_events (session_wa_id, created_at);
```

P1 — CRITICAL order_status_history — Fulfilment funnel timing

Currently impossible to answer: "how long does it take from payment to READY_FOR_PICKUP?" This table makes every status transition measurable. Append a row every time an order status changes.

```
CREATE TABLE order_status_history (
  id              uuid         DEFAULT gen_random_uuid() PRIMARY KEY,
  order_id        text         NOT NULL,
  from_status     text,
  to_status       text         NOT NULL,
  changed_at      timestamptz  DEFAULT now(),
  changed_by_wa_id text
);

CREATE INDEX idx_osh_order  ON order_status_history (order_id, changed_at);
CREATE INDEX idx_osh_status ON order_status_history (to_status, changed_at);
```

P2 — HIGH Column additions to existing tables

```
-- Merchant: add province for geographic analysis
ALTER TABLE "Merchant" ADD COLUMN IF NOT EXISTS province text;
ALTER TABLE "Merchant" ADD COLUMN IF NOT EXISTS acquisition_source text; -- 'invite_form' | 'dir
ALTER TABLE "Merchant" ADD COLUMN IF NOT EXISTS first_order_at timestamptz; -- denorm for fast c

-- MerchantCustomer: geography + denorm spend
ALTER TABLE "MerchantCustomer" ADD COLUMN IF NOT EXISTS province text;
ALTER TABLE "MerchantCustomer" ADD COLUMN IF NOT EXISTS total_spend float DEFAULT 0;
ALTER TABLE "MerchantCustomer" ADD COLUMN IF NOT EXISTS order_count int DEFAULT 0;

-- UserSession: cart as queryable JSONB, not text
ALTER TABLE "UserSession" ALTER COLUMN cart_json TYPE jsonb USING cart_json::jsonb;
CREATE INDEX idx_session_cart ON "UserSession" USING gin(cart_json);

-- ShortLink: track clicks
ALTER TABLE "ShortLink" ADD COLUMN IF NOT EXISTS click_count int DEFAULT 0;
ALTER TABLE "ShortLink" ADD COLUMN IF NOT EXISTS last_clicked_at timestamptz;

-- invite_applications: link to activated merchant
ALTER TABLE invite_applications ADD COLUMN IF NOT EXISTS merchant_id text; -- set on activation
ALTER TABLE invite_applications ADD COLUMN IF NOT EXISTS reviewed_at timestamptz;
ALTER TABLE invite_applications ADD COLUMN IF NOT EXISTS reviewer_note text;
```

P3 — MEDIUM

search_queries — Demand intelligence

Every search the bot processes is currently discarded. Logging them reveals what customers want that merchants don't stock, and which zero-result searches signal catalogue gaps.

```
CREATE TABLE search_queries (
  id              uuid        DEFAULT gen_random_uuid() PRIMARY KEY,
  created_at      timestamptz DEFAULT now(),
  wa_id           text,
  merchant_handle text,       -- null = platform-wide search
  query           text        NOT NULL,
  results_count   int         DEFAULT 0,
  clicked_product_id text     -- null if user abandoned
);

CREATE INDEX idx_sq_query   ON search_queries (lower(query));
CREATE INDEX idx_sq_zero    ON search_queries (results_count) WHERE results_count = 0;
```

05 SQL Views

Five views to run against the existing schema today — no migration required.

View 1 — Merchant Performance Dashboard

```sql
CREATE OR REPLACE VIEW vw_merchant_performance AS
SELECT
  m.id,
  m.trading_name,
  m.store_category,
  m.status,
  m."createdAt"                                          AS joined_at,
  COUNT(DISTINCT o.id)                                   AS total_orders,
  COUNT(DISTINCT CASE WHEN o.status = 'COMPLETED' THEN o.id END) AS completed_orders,
  COALESCE(SUM(CASE WHEN o.status = 'COMPLETED' THEN o.total ELSE 0 END), 0) AS gmv,
  ROUND(AVG(CASE WHEN o.status = 'COMPLETED' THEN o.total END)::numeric, 2)  AS aov,
  COUNT(DISTINCT mc.wa_id)                               AS unique_customers,
  COUNT(DISTINCT p.id)                                   AS active_products,
  COUNT(DISTINCT s.id)                                   AS active_services,
  COUNT(DISTINCT w.wa_id)                                AS wishlist_saves
FROM "Merchant" m
LEFT JOIN "Order"           o  ON o.merchant_id  = m.id
LEFT JOIN "MerchantCustomer" mc ON mc.merchant_id = m.id
LEFT JOIN "Product"         p  ON p.merchant_id  = m.id  AND p.status = 'ACTIVE'
LEFT JOIN "Service"         s  ON s.merchant_id  = m.id  AND s.is_active = true
LEFT JOIN "Wishlist"        w  ON w.product_id   = p.id
GROUP BY m.id;
```

View 2 — Customer Lifetime Value

```sql
CREATE OR REPLACE VIEW vw_customer_ltv AS
SELECT
  o.customer_id                                          AS wa_id,
  COUNT(o.id)                                            AS total_orders,
  COUNT(CASE WHEN o.status = 'COMPLETED' THEN 1 END)     AS completed_orders,
  COALESCE(SUM(CASE WHEN o.status = 'COMPLETED' THEN o.total END), 0) AS lifetime_value,
  ROUND(AVG(CASE WHEN o.status = 'COMPLETED' THEN o.total END)::numeric, 2) AS avg_order_value,
  COUNT(DISTINCT o.merchant_id)                          AS merchants_bought_from
  MIN(o."createdAt")                                     AS first_order_at,
  MAX(o."createdAt")                                     AS last_order_at,
  EXTRACT(epoch FROM (MAX(o."createdAt") - MIN(o."createdAt")))/86400 AS tenure_days
FROM "Order" o
GROUP BY o.customer_id;
```

View 3 — Merchant Cohort (Month of First Order)

```sql
CREATE OR REPLACE VIEW vw_merchant_cohorts AS
WITH first_orders AS (
  SELECT merchant_id, MIN('createdAt') AS first_order_at
  FROM "Order" WHERE status = 'COMPLETED' GROUP BY merchant_id
)
SELECT
  DATE_TRUNC('month', fo.first_order_at)  AS cohort_month,
  m.store_category,
  COUNT(DISTINCT m.id)                     AS merchants_activated,
  AVG(gmv_sub.merchant_gmv)               AS avg_gmv_per_merchant
FROM first_orders fo
JOIN "Merchant" m ON m.id = fo.merchant_id
JOIN (
  SELECT merchant_id, SUM(total) AS merchant_gmv
  FROM "Order" WHERE status = 'COMPLETED' GROUP BY merchant_id
) gmv_sub ON gmv_sub.merchant_id = m.id
GROUP BY 1, 2 ORDER BY 1 DESC;
```

View 4 — Product Performance with Demand Signals

```sql
CREATE OR REPLACE VIEW vw_product_performance AS
SELECT
  p.id,
  p.merchant_id,
  m.trading_name                           AS merchant_name,
  m.store_category,
  p.name,
  p.price,
  p.is_in_stock,
  p.status,
  COALESCE(SUM(oi.quantity), 0)            AS units_sold,
  COALESCE(SUM(oi.quantity * oi.price), 0) AS revenue,
  COUNT(DISTINCT oi.order_id)              AS order_appearances,
  COUNT(DISTINCT w.wa_id)                  AS wishlist_saves,
  p."createdAt"                            AS listed_at,
  p."updatedAt"                            AS last_updated
FROM "Product" p
JOIN "Merchant"   m  ON m.id  = p.merchant_id
LEFT JOIN "OrderItem" oi ON oi.product_id = p.id
LEFT JOIN "Wishlist"   w  ON w.product_id  = p.id
GROUP BY p.id, m.trading_name, m.store_category;
```

View 5 — Invite Pipeline Funnel

```sql
CREATE OR REPLACE VIEW vw_invite_pipeline AS
SELECT
  province,
  category,
  heard_from,
  social_following,
  monthly_orders,
  employees,
  COUNT(*)                                                  AS applications,
```

```sql
  COUNT(CASE WHEN status = 'pending'  THEN 1 END)              AS pending,
  COUNT(CASE WHEN status = 'approved' THEN 1 END)              AS approved,
  COUNT(CASE WHEN status = 'rejected' THEN 1 END)              AS rejected,
  ROUND(100.0 * COUNT(CASE WHEN status = 'approved' THEN 1 END)
    / NULLIF(COUNT(*), 0), 1)                                   AS approval_rate_pct,
  DATE_TRUNC('week', created_at)                                AS week
FROM invite_applications
GROUP BY 1, 2, 3, 4, 5, 6, 9;
```

06

Insights & Features Unlocked

What the data tells us — and how we use it to improve the experience.

MERCHANT SUCCESS SCORING

Using `vw_merchant_performance`, score each merchant weekly on a simple composite: GMV growth + repeat customer rate + catalogue freshness (updatedAt recency). Low scorers get proactive outreach from the Omeru team before they churn.

- Flag merchants with 0 completed orders in 14 days
- Flag merchants with out-of-stock rate > 50%
- Celebrate merchants hitting GMV milestones via WhatsApp

DEMAND VS SUPPLY GAP ANALYSIS

Products on wishlists that are out of stock represent money left on the table. Cross-referencing `Wishlist` with `Product.is_in_stock = false` gives a ranked list of restocking priorities to share with merchants.

- Notify merchant when a wishlisted product goes out of stock
- Notify customer when a wishlisted product restocks
- Surface high-wishlist products in the Browse UI

CATEGORY INTELLIGENCE (INVITE PIPELINE)

The `invite_applications` table already captures category, province, social following, and heard_from. This is market research at zero marginal cost. Weekly roll-ups reveal which categories are over-applied vs under-served on the platform.

- Which province has the most unmet Food & Drink demand?
- Does Instagram or TikTok send higher-quality applicants?
- Do solo traders or 2–5 person teams perform better?

CROSS-POLLINATION SCORE

Omeru's value to merchants includes cross-merchant customer exposure. Track how many customers a new merchant inherited from the broader Omeru customer base vs acquired independently. This is a key pitch metric.

- Customers shared across ≥2 merchants
- Time from merchant activation to first cross-merchant customer
- Category affinity pairs (e.g. Fashion + Beauty)

FULFILMENT SPEED BENCHMARKING

Once `order_status_history` is in place, measure the median time from PAID → READY_FOR_PICKUP per merchant category. Share benchmarks back to merchants: "Your average prep time is 47 min. Top Food & Drink merchants are at 22 min."

- Median fulfilment time by category
- Orders abandoned after payment (PAID → CANCELLED)

ZERO-RESULT SEARCH MINING

Once `search_queries` is logging, filter for `results_count = 0`. These are the clearest possible signals of unmet demand. Review weekly and use as the primary input for merchant recruitment targeting — find merchants who sell what customers can't find.

- Top 20 zero-result search terms per week
- Category of failed searches → recruit in that category

- Alert when order sits in PENDING > 30 min

- Geographic zero-result clusters → recruit in that province

Booking & Service Insights

Services via the Booking model have rich temporal data (start_at, end_at, status) that products don't have. This unlocks time-based analysis unique to service merchants:

QUESTION	SQL APPROACH
What hour of day are most bookings made?	EXTRACT(hour FROM "createdAt") GROUP BY hour
What day of week has highest rejection rate?	EXTRACT(dow FROM start_at) WHERE status = 'REJECTED'
Which services have the longest booking lead time?	AVG(start_at - "createdAt") per service_id
Are customers rebooking the same service?	COUNT per (customer_wa_id, service_id) > 1
Revenue at risk from pending bookings?	SUM(s.price) WHERE b.status = 'CONFIRMED' AND paid = false

07 Implementation Roadmap

Prioritised by impact-to-effort. P1 items take under a day each.

PRIORITY	ACTION	EFFORT	UNLOCKS
P1	Run the 5 SQL views in Supabase	1 hour	Immediate analytical access to all existing data
P1	Create `platform_events` table + indexes	2 hours	Behavioural funnel, product views, search
P1	Create `order_status_history` table	2 hours	Fulfilment speed metrics, SLA monitoring
P1	Cast `UserSession.cart_json` to JSONB	30 min	Cart abandonment rate, basket size analysis
P2	Add `province` to Merchant table	30 min	Geographic GMV heat maps, regional recruitment
P2	Add `click_count` to ShortLink	2 hours incl. backend	Link-level attribution, campaign ROI
P2	Add `merchant_id` FK to `invite_applications`	1 hour	Closed-loop pipeline — track application → merchant → GMV
P2	Denorm `order_count` + `total_spend` on MerchantCustomer	3 hours incl. backfill	Fast LTV lookups without full Order scan
P3	Create `search_queries` table + log all bot searches	4 hours	Zero-result mining, catalogue gap analysis
P3	Add AuditLog GIN index on `metadata_json`	10 min	Fast audit log queries by entity type
P3	Set up Metabase or Evidence.dev on top of Supabase	1 day	Self-serve dashboards without writing SQL
P4	Materialise `vw_merchant_performance` as nightly table	2 hours	Sub-second dashboard queries at scale

Recommended BI tool

Supabase already exposes a direct Postgres connection. The fastest path to dashboards for non-technical stakeholders is **Metabase** (open source, runs as a Docker container or on Metabase Cloud). It connects

directly to Supabase via the connection string, and the views defined in section 05 become ready-made "models" that anyone can filter, group, and chart without writing SQL.

> **Alternative:** Evidence.dev generates static analytics sites from SQL — good if you want dashboards in version control. Superset (Apache) is the enterprise option. All three connect to Supabase Postgres out of the box.